The Agent Reliability Stack: Six Single-Concern, Zero-Dependency Libraries for LLM Agents

Mukunda Rao Katta
Independent Researcher
mukunda.vjcs6@gmail.com

ABSTRACT

Reliability concerns for large-language-model (LLM) agents are typically addressed inside frameworks that bundle prompting, tool routing, and runtime governance into a single dependency. We present the *agent-stack*: six small, single-concern reliability libraries published independently to npm, PyPI, and the Model Context Protocol (MCP) registry. The libraries are AgentFit (context-window fitting), AgentGuard (network egress allowlist), AgentSnap (snapshot tests for tool-call traces), AgentVet (validate tool arguments before execution), AgentCast (structured-output enforcer), and AgentBudget (token and dollar caps). Each library is zero-dependency, has a TypeScript implementation with hand-maintained type declarations, a Python port with the same surface, and an MCP-server variant. Across the eighteen public packages we identify a recurring artifact pattern: a single class or function as the public surface, a typed error carrying retry-friendly context, and an opt-in automatic adapter for popular provider response shapes. The contribution is a documented design pattern for reliability primitives that compose by inclusion rather than by framework lock-in. Tool URL: https://mukundakatta.github.io/agent-stack/. Screencast: https://www.youtube.com/watch?v=AGENT_STACK_DEMO[1].

KEYWORDS

LLM agents, reliability primitives, agent infrastructure, snapshot testing, schema validation, structured output, budget caps, zero-dependency libraries, Model Context Protocol

1 INTRODUCTION

LLM agent runtimes are typically distributed as monolithic frameworks. A single dependency provides prompting, tool dispatch, retries, schema validation, observability, budget enforcement, and safety filters. This packaging is convenient when an agent matches the framework's assumptions. It becomes a liability when the agent does not: framework upgrades pull in unrelated changes, opinionated defaults are difficult to override, and the operator cannot adopt one reliability concern without adopting the others.

We present the agent-stack as a deliberate counterpoint. It splits reliability into six independent libraries. Each library solves one problem, has zero runtime dependencies, and can be adopted in isolation. Composition happens by including the libraries the operator needs, not by inheriting framework defaults. The artifact described in this paper is the eighteen-package public release of the stack on npm, PyPI, and the MCP registry; the tool URL above resolves to the landing page that links every package, the source code, the test suites, and the published preprint with DOI `10.5281/zenodo.20074702`.

[1]Screencast URL will be replaced with the final upload before the camera-ready deadline.

2 THE SIX PRIMITIVES

2.1 AgentFit

Concern: fit a list of messages into a model's context window. The library accepts a token-aware budget, a list of messages, and a strategy. Bundled strategies are drop-oldest, drop-middle, and priority-keyed. The output is a fitted message list and a structured trim report describing which messages were dropped and why. Principal failure mode addressed: silent context truncation; AgentFit surfaces dropped messages explicitly rather than letting them disappear from the input.

2.2 AgentGuard

Concern: declarative network-egress allowlist around agent tools. Tool functions are wrapped so that any outbound HTTP request to a domain not on the allowlist throws with the offending URL and the active allowlist. Principal failure mode addressed: a tool that fetches a URL the operator did not anticipate; the failure is caught at the egress site rather than discovered later in billing or data-flow audits.

2.3 AgentSnap

Concern: snapshot tests for tool-call traces. Tool functions are wrapped with `traceTool`; during a recorded run the library captures the sequence of tool names, argument hashes, and result hashes into a stable JSON envelope. A baseline file is committed alongside the test, and subsequent test runs compare the new trace to the baseline and report drift in a human-readable form. Principal failure mode addressed: undetected behavior change in the tool sequence after a model or prompt revision.

2.4 AgentVet

Concern: validate tool-call arguments before the tool function runs. Each tool is wrapped with a schema, a Zod validator, or a small built-in shape spec. Calls that fail validation throw a typed `ToolArgError` that carries an LLM-friendly retry hint formatted for direct insertion into the next conversation turn. Principal failure mode addressed: a model that hallucinates argument types and reaches the tool body.

2.5 AgentCast

Concern: enforce structured output on LLM completions. The library wraps an LLM call with a validate-and-retry loop: the response is parsed, validation errors are translated into a feedback message, and the LLM is asked to correct itself up to a configurable

retry limit. Principal failure mode addressed: a model that returns prose where the application expects a structured object.

2.6 AgentBudget

Concern: track token and dollar usage across an agent run and refuse calls that would push past a configured cap. The user constructs a `Budget` with caps for input tokens, output tokens, total tokens, dollars, or any combination. After each LLM call the user records usage; the library raises `BudgetExceededError` carrying the offending cap, the limit, the attempted total, and the model name. A built-in pricing table covers the dominant Anthropic and OpenAI models. Principal failure mode addressed: a planner loop that accidentally calls a model thousands of times.

3 CROSS-CUTTING INVARIANTS

The six libraries share three deliberate invariants that together define the artifact pattern.

Single noun, single verb. AgentFit is a *fit* over messages. AgentGuard is a *guard* over fetch sites. AgentSnap is a *snap* over tool traces. AgentVet is a *vet* over tool arguments. AgentCast is a *cast* over LLM outputs. AgentBudget is a *record* and a *cap* over LLM usage. The naming convention is part of the artifact: it tells the operator which library to reach for without consulting a manual.

Every primitive throws. There is no silent fallback path. The libraries refuse to pretend a violated invariant is fine; the operator adopted them precisely to make a specific failure observable.

Errors are typed classes with named fields. `BudgetExceededError` carries `cap`, `limit`, `attempted`, `overshoot`, and `model`. `ToolArgError` carries the tool name, the validation error, the rejected arguments, and a `toLLMFeedback()` method. Errors are designed to be programmatically dispatched and human-readable in incident logs simultaneously.

4 CROSS-LANGUAGE SYMMETRY

Each library exists in three implementations: TypeScript, Python, and an MCP server. The TypeScript implementation is the reference. The Python port mirrors the TypeScript surface with snake-case names. The MCP server exposes the same primitives over the Model Context Protocol so that any MCP-aware client (Claude Desktop, Cursor, Cline, Windsurf, Zed) can adopt the reliability surface declaratively from a configuration file rather than from agent code. The cost of the symmetry is that each fix must be applied three times. The benefit is that the design pattern, not any single implementation, is the unit of distribution.

5 COMPOSITION AND USAGE

The six primitives are designed to compose by inclusion. The conceptual pipeline runs left to right: *fit* messages into context, *guard* tool egress, *snap* tool traces against a baseline, *vet* tool arguments before execution, *cast* LLM responses to typed data, and *cap* token and dollar usage. The pipeline is illustrative, not enforced; each library is independent and can be adopted in isolation.

A minimal AgentBudget integration is shown below; the same idiom applies to the other five primitives:

```
import { Budget } from '@mukundakatta/agentbudget';

const budget = new Budget({
  maxTotalTokens: 200_000,
  maxCostUsd: 5.00,
});

for (const turn of turns) {
  const r = await client.messages.create({...});
  budget.recordUsage({
    model: r.model,
    inputTokens: r.usage.input_tokens,
    outputTokens: r.usage.output_tokens,
  });
}
```

Because the libraries are zero-dependency, composing them does not introduce transitive constraints. An operator can pin AgentBudget at one major version and AgentSnap at a different major version without conflict resolution. This is a deliberate trade against the convenience of a single-import framework: the operator pays a small cognitive cost in exchange for a trust surface they can audit one library at a time.

6 EVALUATION PATH

The artifact is evaluated at four layers. (1) *Artifact-level*: every package on npm passes `npm test` on a fresh checkout under Node 18+ and reports zero runtime dependencies under `npm ls --omit=dev`; every PyPI port passes `pytest` on Python 3.10–3.13 with no installed dependencies beyond the standard library. (2) *Integration-level*: each primitive composes with realistic provider SDK shapes (Anthropic, OpenAI, Groq) without surprise, verified through the integration tests in each repository. (3) *Incident-level*: when a primitive fires, the structured error carries enough context for the operator to fix the underlying cause without reading the library's source. (4) *Longitudinal*: across model and provider revisions, the primitives remain useful because the surfaces they wrap (token usage, tool call traces, structured output, network egress) are stable concerns even as model behavior drifts.

The reproducibility package on Zenodo (DOI `10.5281/zenodo.20074702`) bundles the manuscript source, the rendered preprint PDF, the LaTeX bundle, abstracts and keywords, and a build script that regenerates the deposit from `paper.md` via `reportlab`. All eighteen packages are MIT-licensed; the manuscript is CC-BY-4.0.

7 LIMITATIONS

Three limitations are explicit. The agent-stack does not include a primitive for rate limiting; that interacts with provider headers and retry policies outside the scope of the six current libraries. It does not provide a unified observability surface; operators who want a single trace must compose one. It does not include a primitive for memory or session safety; that overlaps with personal-assistant architecture (the companion paper *Karna* [1]) rather than reliability primitives.

8 AVAILABILITY

- Landing page: https://mukundakatta.github.io/agent-stack/
- Source repositories: https://github.com/MukundaKatta (`Agent*`)
- TypeScript: `@mukundakatta/agentfit`, `agentguard`, `agentsnap`, `agentvet`, `agentcast`, `agentbudget` on npm

- Python: `agentfit-py`, `agentguard-firewall`, `agentsnap-py`, `agentvet-py`, `agentcast-py`, `agentbudget-py` on PyPI
- MCP servers: `*-mcp` variants on npm and the MCP registry
- Preprint: https://doi.org/10.5281/zenodo.20074702
- Companion paper repository: https://github.com/MukundaKatta/agent-stack-reliability-primitives-paper

REFERENCES

[1] Mukunda Rao Katta. Karna: A Chat-Native, Multi-Channel Architecture for Personal AI Chief-of-Staff Agents. Zenodo, 2026. DOI: 10.5281/zenodo.20074229.
[2] Anthropic. Model Context Protocol Specification. https://modelcontextprotocol.io/, accessed 2026-05-07.
[3] Harrison Chase et al. LangChain. https://www.langchain.com/, accessed 2026-05-07.
[4] Jerry Liu et al. LlamaIndex. https://www.llamaindex.ai/, accessed 2026-05-07.
[5] Microsoft. AutoGen: Enable Next-Gen Large Language Model Applications. https://microsoft.github.io/autogen/, accessed 2026-05-07.
[6] OpenAI. OpenAI Agents SDK. https://openai.github.io/openai-agents-python/, accessed 2026-05-07.
[7] Julien Danjou. Tenacity: Retrying library for Python. https://github.com/jd/tenacity, accessed 2026-05-07.
[8] Samuel Colvin et al. Pydantic. https://docs.pydantic.dev/, accessed 2026-05-07.
[9] Tom Christie. HTTPX: A next generation HTTP client for Python. https://www.python-httpx.org/, accessed 2026-05-07.